

CHƯƠNG 2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1. Mô tả bài toán

Sinh viên ngành Công nghệ thông tin trong quá trình học môn Cấu trúc dữ liệu và Giải thuật phải làm việc với khối lượng tài liệu lớn gồm giáo trình, slide bài giảng, sách tham khảo ở định dạng PDF hoặc DOCX với hàng trăm đến hàng nghìn trang. Nhu cầu tra cứu nhanh một khái niệm trong tập tài liệu cá nhân là nhu cầu thường xuyên nhưng chưa được đáp ứng tốt bởi các công cụ hiện có. Các giải pháp chatbot AI phổ biến như ChatGPT hay Gemini tuy có thể trả lời câu hỏi về DSA, nhưng lại tồn tại hai hạn chế nghiêm trọng trong bối cảnh học tập: chúng không làm việc được với tài liệu nội bộ của người dùng, và phụ thuộc hoàn toàn vào kết nối internet cùng API trả phí. Ngoài ra, việc tải tài liệu học tập lên các dịch vụ đám mây còn tiềm ẩn rủi ro về bảo mật và quyền riêng tư.

Kỹ thuật RAG mở ra hướng giải quyết phù hợp, cho phép mô hình ngôn ngữ lớn trả lời câu hỏi dựa trực tiếp trên tài liệu do người dùng cung cấp. Tuy nhiên, một thành phần then chốt trong pipeline RAG là Chunking - bước phân đoạn tài liệu trước khi vector hóa vẫn chưa được nghiên cứu đầy đủ về mức độ ảnh hưởng đến chất lượng hệ thống, đặc biệt trên tài liệu kỹ thuật DSA trong môi trường hoàn toàn offline. Chính khoảng trống này đặt ra hai bài toán nghiên cứu có liên kết chặt chẽ với nhau.

Bài toán tập trung vào việc đánh giá định lượng ảnh hưởng của Chunking đến hiệu năng Retrieval. Đầu vào là tập tài liệu DSA ở định dạng PDF hoặc DOCX, cùng với một bộ câu hỏi benchmark có ground truth và ma trận cấu hình thực nghiệm gồm năm phương pháp Chunking kết hợp với ba mức chunk size khác nhau. Đầu ra cần đạt được là bộ metric định lượng bao gồm Precision@5, Recall@5, F1@5, Hit Rate@5, MRR cho từng cấu hình, từ đó rút ra kết luận về phương pháp Chunking tối ưu. Trong thiết kế thực nghiệm này, phương pháp Chunking và chunk_size đóng vai trò biến độc lập, các metric là biến phụ thuộc, còn LLM, embedding model và reranker được giữ cố định như các biến kiểm soát.

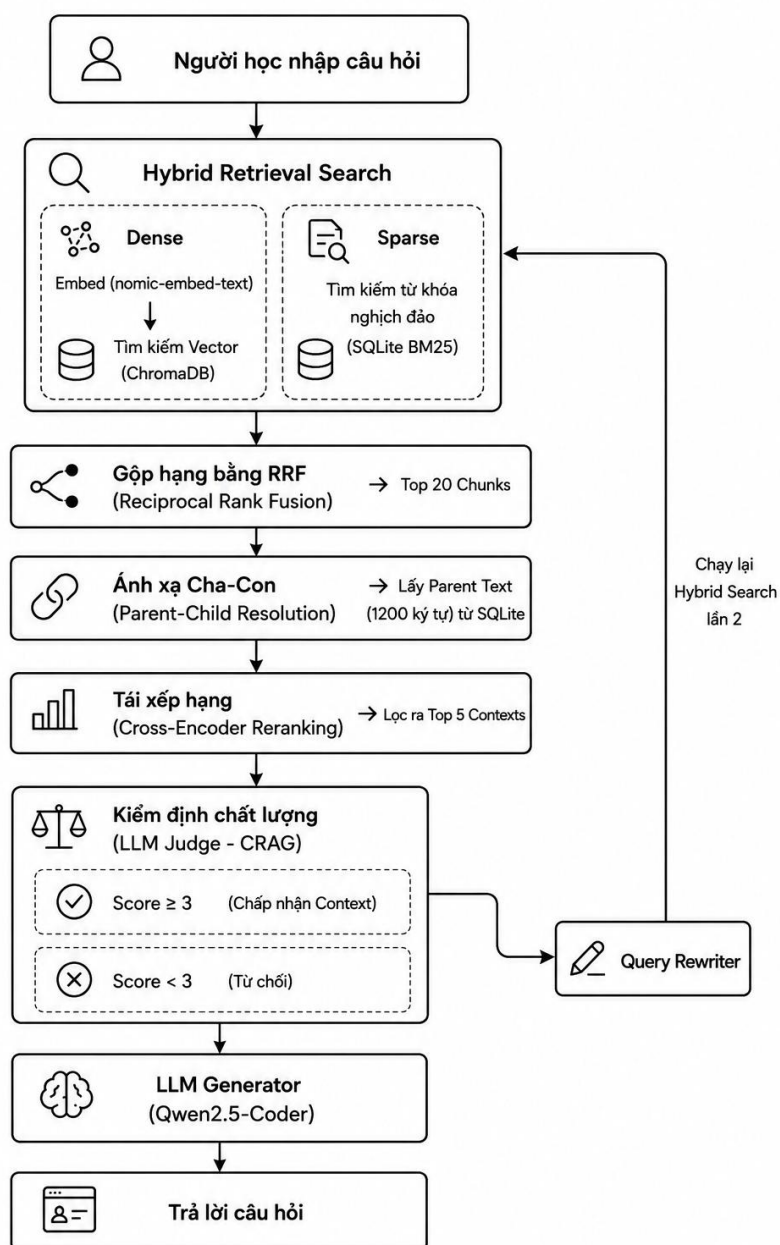
Bài toán còn lại là xây dựng ứng dụng học tập DSA hoạt động hoàn toàn offline. Người dùng cung cấp tài liệu DSA và đặt câu hỏi bằng ngôn ngữ tự nhiên, hệ thống trả về câu trả lời chính xác dựa trên tài liệu kèm theo trích dẫn nguồn gồm tên file và số trang, thông qua giao diện Desktop.

Để định hướng nghiên cứu, đề tài đặt ra ba giả thuyết cần được kiểm chứng qua thực nghiệm. Thứ nhất, phương pháp Chunking có ảnh hưởng có ý nghĩa thống kê đến hiệu năng Retrieval, đo bằng $F1@5$, trên tài liệu kỹ thuật DSA. Thứ hai, không có phương pháp Chunking nào vượt trội tuyệt đối trên tất cả metric, bởi sự tối ưu phụ thuộc vào từng loại câu hỏi cụ thể như fact lookup, phân tích hay suy luận đa bước. Thứ ba, các phương pháp Chunking tôn trọng ranh giới ngữ nghĩa như Semantic hay Parent-Child sẽ cho hiệu năng cao hơn so với cách cắt đứt tùy ý theo kích thước cố định, đặc biệt trên tài liệu DSA có cấu trúc đa dạng kết hợp văn xuôi, code và bảng biểu.

Về phía hệ thống, ba mục tiêu cụ thể được đặt ra là xây dựng thực nghiệm có thể tái lập để đánh giá định lượng hiệu năng Retrieval theo từng chiến lược Chunking trên bộ câu hỏi benchmark. Hệ thống hướng tới ba nhóm người dùng chính: sinh viên đại học đang học môn DSA có nhu cầu tra cứu tài liệu nhanh, giảng viên muốn tích hợp tài liệu giảng dạy vào hệ thống hỏi đáp, và nhà nghiên cứu quan tâm đến việc đánh giá các chiến lược Chunking trên tài liệu kỹ thuật.

Việc hiện thực hóa hệ thống đặt ra một số thách thức kỹ thuật không nhỏ. Ràng buộc VRAM hạn chế buộc nhóm phải sử dụng reranker chạy trên CPU và giới hạn ngữ cảnh đầu vào ở mức tối đa bốn chunk. Tính đa dạng của tài liệu DSA - vừa có văn xuôi, vừa có đoạn code, vừa có bảng biểu - đòi hỏi phải thử nghiệm nhiều chiến lược Chunking khác nhau thay vì áp dụng một phương pháp duy nhất.

2.2. Phân tích yêu cầu hệ thống



Hình 2.1. Sơ đồ hệ thống

Pipeline xử lý câu hỏi của hệ thống RAG với sáu bước tuần tự, kết hợp Hybrid Retrieval và cơ chế tự đánh giá chất lượng tài liệu trước khi sinh câu trả lời. Toàn bộ pipeline hoạt động hoàn toàn offline, dựa trên hai kho lưu trữ cục bộ là Vector Store (ChromaDB) và Metadata Store (SQLite).

Bước 1 Hỏi đáp: Người dùng nhập câu hỏi bằng ngôn ngữ tự nhiên thông qua giao diện chat. Câu hỏi có thể là dạng tra cứu khái niệm, yêu cầu giải thích thuật toán, so sánh cấu trúc dữ liệu hoặc hỏi về độ phức tạp thời gian. Đây là điểm khởi đầu của toàn bộ pipeline và là đầu vào duy nhất từ phía người dùng.

Bước 2 Hybrid Retriever: Sau khi nhận được câu hỏi từ người dùng, hệ thống sẽ truy xuất tài liệu bằng cơ chế Hybrid Retrieval, kết hợp hai phương pháp tìm kiếm là tìm kiếm từ khóa và tìm kiếm ngữ nghĩa. Nhánh Dense Retrieval sử dụng mô hình nomic-embed-text để chuyển câu hỏi thành vector embedding và thực hiện tìm kiếm độ tương đồng trong Vector Store, giúp phát hiện các đoạn tài liệu có ý nghĩa gần với câu hỏi ngay cả khi không chứa các từ khóa giống nhau. Còn nhánh Sparse Retrieval sử dụng thuật toán BM25 được xây dựng trên Metadata Store để tìm kiếm dựa trên tần suất và mức độ quan trọng của các từ khóa, đặc biệt hiệu quả đối với các truy vấn chứa thuật ngữ chuyên ngành, tên thuật toán hoặc ký hiệu kỹ thuật. Kết quả từ hai nhánh được hợp nhất bằng thuật toán Reciprocal Rank Fusion, tạo ra danh sách 20 chunk có mức độ liên quan cao nhất, đồng thời giảm sự phụ thuộc vào một phương pháp truy xuất đơn lẻ.

Bước 3 Parent–Child Resolution và Cross-Encoder Reranking: Danh sách các chunk thu được từ bước Hybrid Retrieval gồm các child chunk có kích thước nhỏ nhằm tối ưu khả năng tìm kiếm ngữ nghĩa. Sau đó, hệ thống thực hiện cơ chế Parent-Child Resolution, sử dụng thông tin ánh xạ được lưu trong SQLite để truy xuất parent text tương ứng của từng child chunk. Mỗi parent text có kích thước khoảng 1.200 ký tự, giúp khôi phục đầy đủ nội dung của đoạn văn gốc mà vẫn giữ được tính liên quan của kết quả truy xuất. Sau đó, các parent text được đưa qua mô hình Cross-Encoder Reranking để đánh giá lại mức độ liên quan giữa toàn bộ ngữ cảnh và câu hỏi. Không giống các phương pháp tính độ tương đồng embedding, Cross-Encoder mã hóa đồng thời câu hỏi và tài liệu trong cùng một lần suy luận, cho phép nắm bắt tốt hơn mối quan hệ ngữ nghĩa giữa hai chuỗi văn bản. Dựa trên điểm số thu được, hệ thống sắp xếp lại thứ hạng và lựa chọn top 5 context có chất lượng cao nhất để chuyển sang bước kiểm định tài liệu.

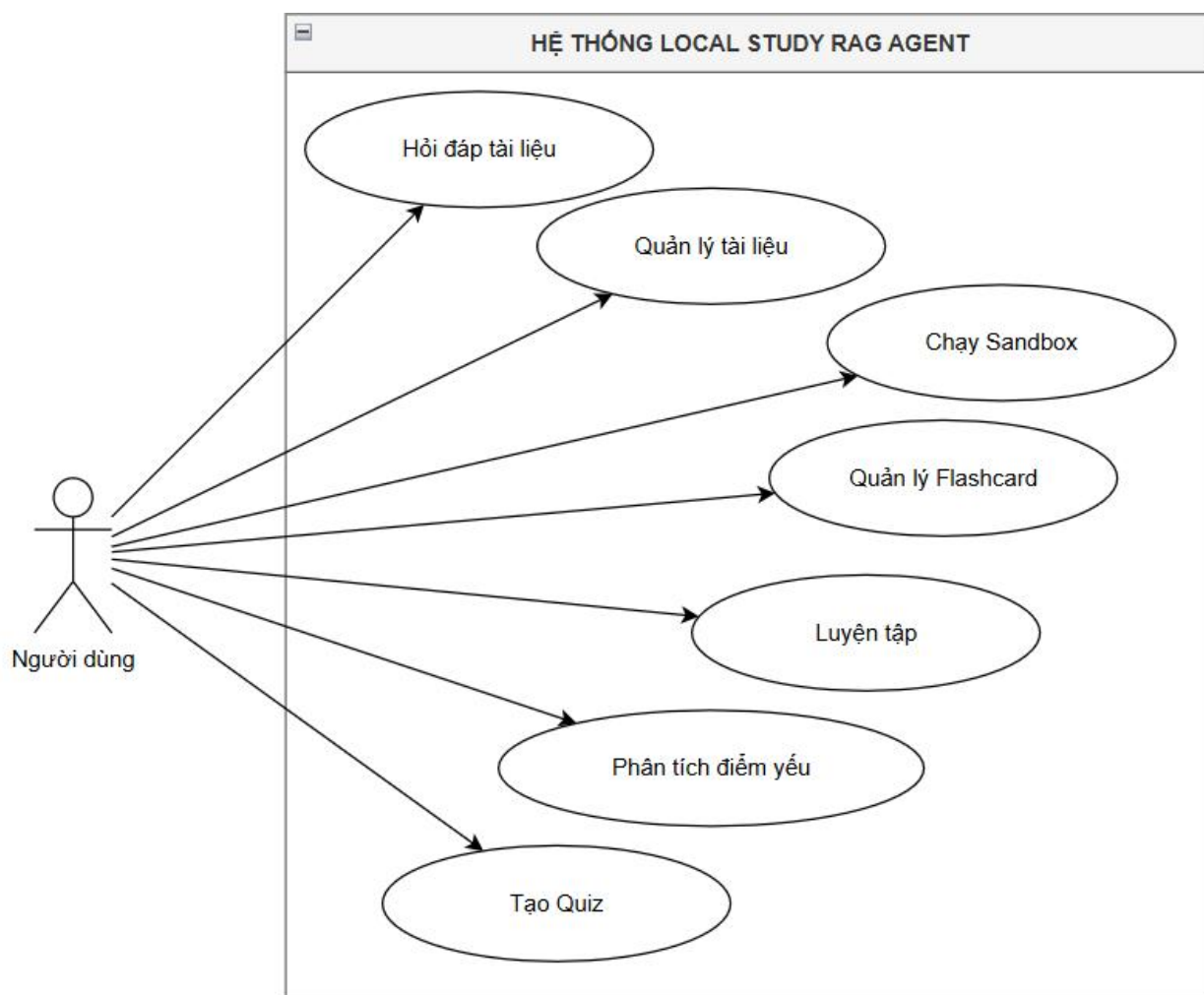
Bước 4 Document Grader: Trước khi đưa tài liệu vào LLM, hệ thống thực hiện bước đánh giá chất lượng tài liệu ứng viên thông qua cơ chế Corrective RAG . Document Grader chấm điểm mức độ liên quan của từng chunk thu được so với câu hỏi gốc, lọc bỏ những chunk không đủ liên quan. Nếu tập tài liệu sau khi lọc không đủ chất lượng để trả lời câu hỏi, hệ thống kích hoạt cơ chế Query Rewrite - tự động diễn đạt lại câu hỏi theo cách khác và lặp lại bước Retrieval để tìm kiếm tài liệu phù hợp hơn. Metadata Store SQLite được tham chiếu tại bước này để bổ sung thông tin nguồn gốc cho các chunk vượt qua ngưỡng đánh giá. Cơ chế tự sửa lỗi này giúp hệ thống tránh được tình huống LLM sinh câu trả lời dựa trên ngữ cảnh kém chất lượng, vốn là nguyên nhân phổ biến dẫn đến hallucination trong các hệ thống RAG đơn giản.

Bước 5 Query Rewrite: Khi Document Grader đánh giá rằng tập ngữ cảnh truy xuất có chất lượng không đủ để trả lời câu hỏi, hệ thống sẽ kích hoạt cơ chế Query Rewrite. Thành phần này sử dụng mô hình ngôn ngữ để diễn đạt lại câu hỏi của người dùng theo cách rõ ràng, đầy đủ ngữ nghĩa hơn, đồng thời vẫn giữ nguyên ý định ban đầu của truy vấn. Việc viết lại câu hỏi giúp bổ sung các thuật ngữ còn thiếu, loại bỏ cách diễn đạt mơ hồ hoặc tối ưu hóa từ khóa nhằm tăng khả năng tìm kiếm của cả hai nhánh Dense Retrieval và Sparse Retrieval. Sau khi tạo truy vấn mới, hệ thống quay trở lại bước Hybrid Retrieval Search và thực hiện toàn bộ quá trình truy xuất, hợp nhất kết quả, ánh xạ Parent-Child và tái xếp hạng một lần nữa. Cơ chế phản hồi này giúp hệ thống nâng cao khả năng tìm được ngữ cảnh phù hợp trong trường hợp truy vấn ban đầu chưa đủ rõ ràng, đồng thời giảm nguy cơ sinh câu trả lời thiếu chính xác do sử dụng tài liệu không liên quan.

Bước 6 LLM Answer Generator: Tập chunk đã qua kiểm định được ghép thành ngữ cảnh và đưa vào mô hình ngôn ngữ Qwen2.5-coder 7B cùng với câu hỏi gốc. Mô hình này được lựa chọn vì khả năng xử lý tốt cả văn bản kỹ thuật lẫn đoạn code - hai thành phần đặc trưng của tài liệu DSA. Toàn bộ quá trình suy luận diễn ra cục bộ trên GPU của máy người dùng mà không gửi dữ liệu ra ngoài, đảm bảo tính riêng tư và khả năng hoạt động offline. LLM được yêu cầu sinh câu trả lời trung thành với ngữ cảnh được cung cấp, hạn chế tối đa việc bổ sung thông tin ngoài tài liệu.

Bước 7 Answer + Citations: Đầu ra cuối cùng được trả về người dùng gồm hai phần: câu trả lời được trình bày rõ ràng bằng ngôn ngữ tự nhiên, và danh sách trích dẫn nguồn ghi rõ tên tài liệu cùng số trang tương ứng cho mỗi đoạn thông tin được sử dụng. Người dùng có thể đối chiếu câu trả lời với tài liệu gốc dựa trên thông tin trích dẫn này. Kết quả sau đó được hiển thị trên giao diện chat, khép lại một vòng xử lý và sẵn sàng cho câu hỏi tiếp theo của người dùng.

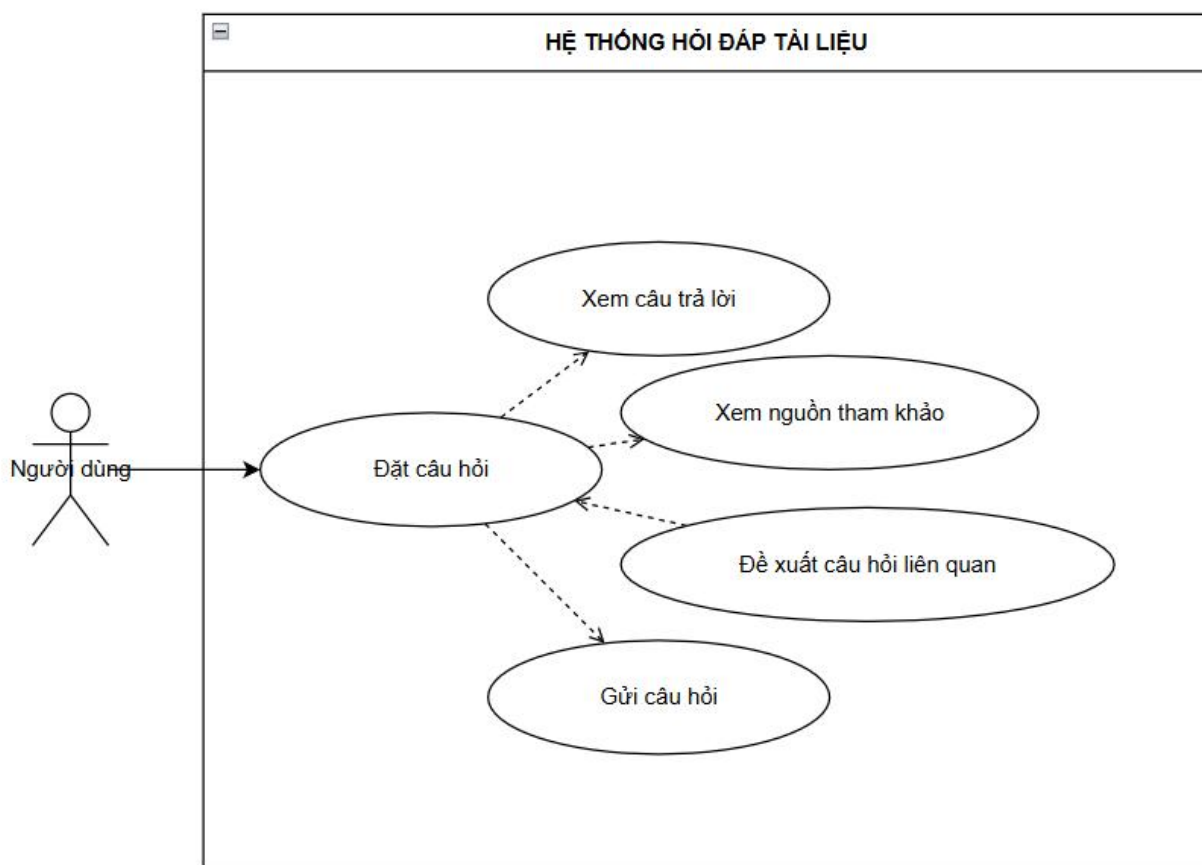
2.3. Biểu đồ Use Case



Hình 2.2. Biểu đồ Use case tổng quát

Hệ thống trợ lý học tập thông minh được thiết kế phục vụ một tác nhân duy nhất là người dùng với 7 chức năng chính được thể hiện trong Hình 3.1. Hỏi đáp tài liệu là chức năng cốt lõi, cho phép người dùng đặt câu hỏi về nội dung DSA và nhận câu trả lời kèm

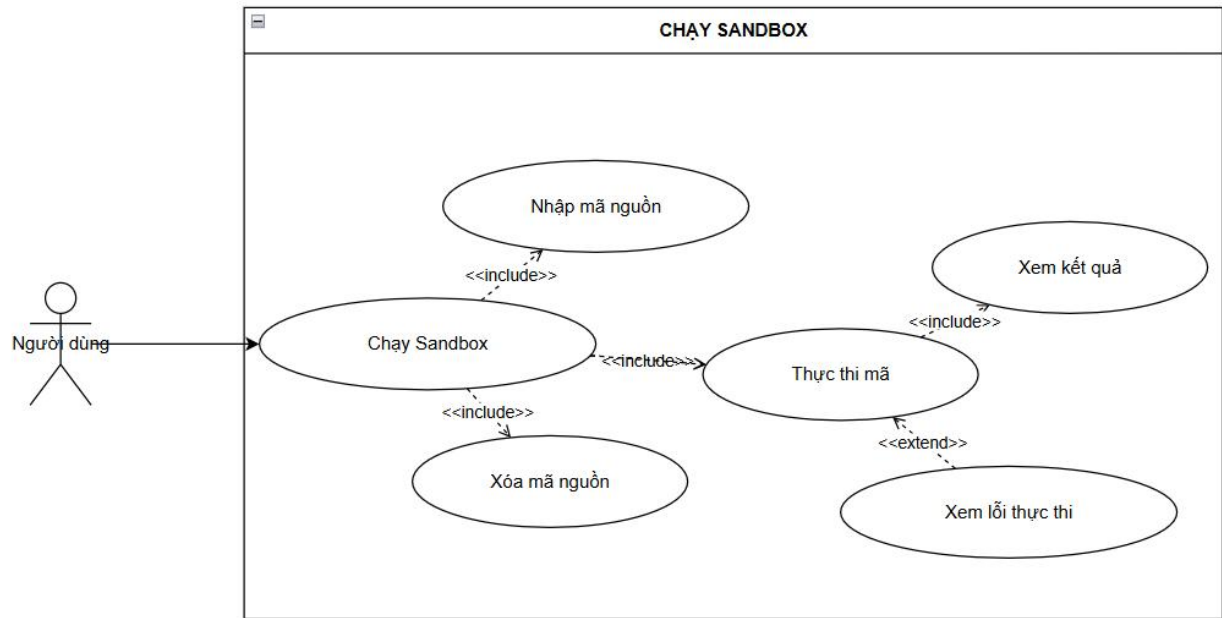
trích dẫn nguồn từ tài liệu đã nạp. Quản lý tài liệu hỗ trợ tải lên, xem danh sách và xóa các tài liệu PDF/DOCX phục vụ hệ thống hỏi đáp. Chạy Sandbox cung cấp môi trường thực thi code C/C++ và Python an toàn ngay trong ứng dụng. Quản lý Flashcard cho phép tạo và ôn tập thẻ ghi nhớ về các khái niệm DSA. Luyện tập cung cấp bài tập lập trình DSA với chấm điểm tự động qua test case. Phân tích điểm yếu tổng hợp lịch sử làm bài để xác định các chủ đề còn yếu và gợi ý ôn tập phù hợp. Tạo Quiz sinh bộ câu hỏi trắc nghiệm theo chủ đề từ nội dung tài liệu đã nạp, kết quả được lưu lại để theo dõi tiến trình học tập.



Hình 2.3. Biểu đồ Use case hỏi đáp tài liệu

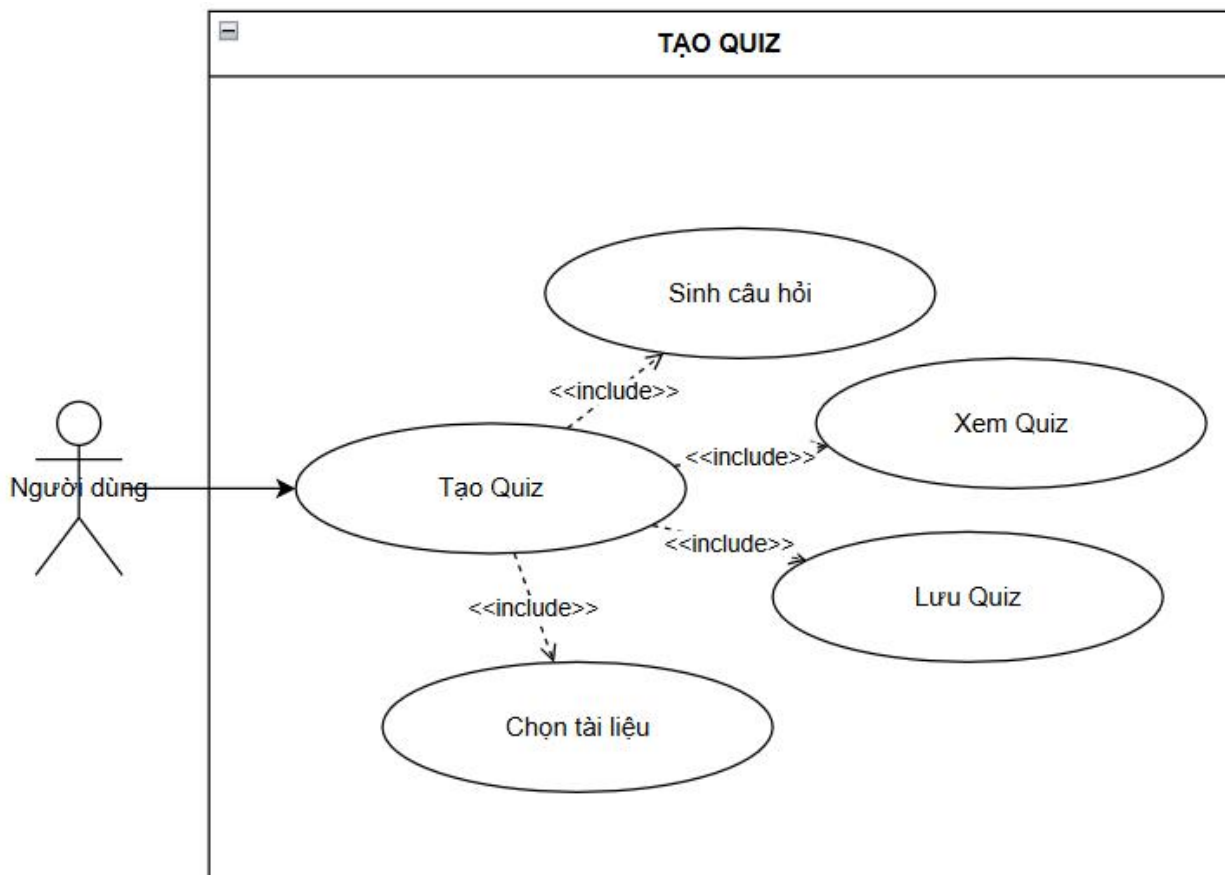
Sau khi đã có ít nhất một tài liệu trong hệ thống, người dùng nhập câu hỏi bằng ngôn ngữ tự nhiên vào ô chat ở phần Cuộc trò chuyện. Hệ thống thực hiện Retrieval trên tập tài liệu đã được vector hóa, tìm kiếm các đoạn văn bản liên quan nhất, sau đó đưa ngữ cảnh thu được vào LLM để sinh câu trả lời. Kết quả trả về gồm phần trả lời chi tiết và

danh sách nguồn trích dẫn ghi rõ tên tài liệu cùng số trang tương ứng. Toàn bộ quá trình diễn ra hoàn toàn offline mà không cần kết nối internet.



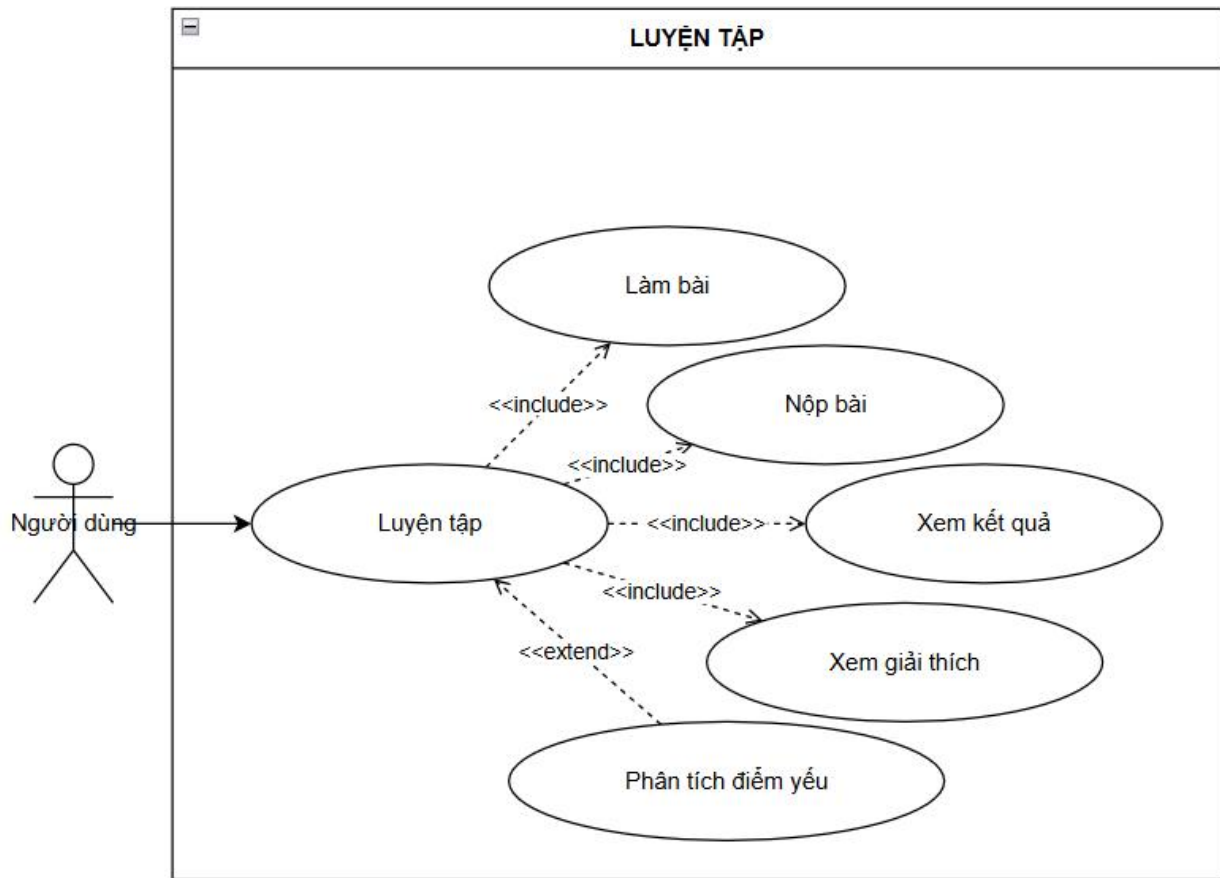
Hình 2.4. Biểu đồ Use case sandbox

Người dùng truy cập chức năng Code Sandbox để viết và thực thi code trực tiếp trong ứng dụng. Người dùng chọn ngôn ngữ lập trình, nhập code vào trình soạn thảo, cung cấp dữ liệu đầu vào nếu cần và nhấn nút chạy. Hệ thống biên dịch và thực thi code, hiển thị kết quả Output cùng thời gian chạy thực tế. Người dùng có thể bổ sung các test case để kiểm tra tính đúng đắn của giải thuật, hệ thống tự động so sánh kết quả thực tế với kết quả mong đợi và đánh dấu pass hoặc fail cho từng test case.



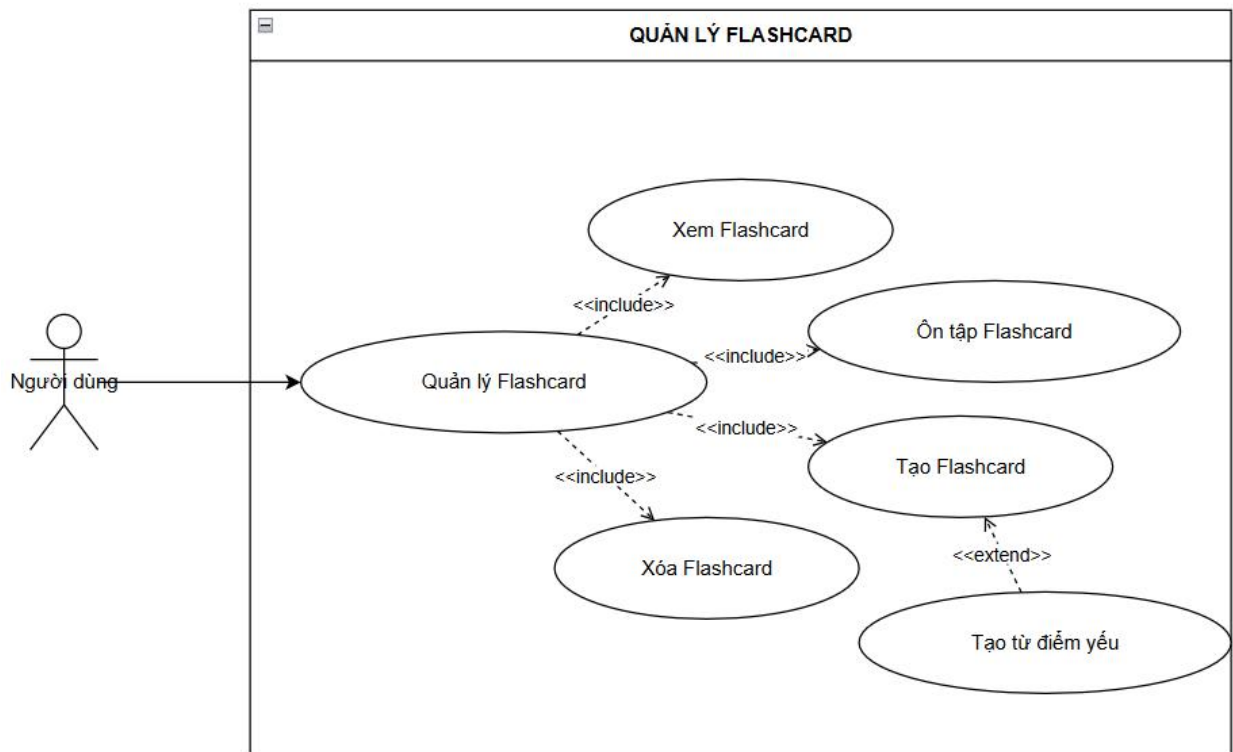
Hình 2.5. Biểu đồ Use case tạo quiz

Người dùng truy cập module Quiz, chọn chủ đề, số câu hỏi và mức độ khó rồi nhấn nút tạo quiz. Hệ thống tự động sinh bộ câu hỏi trắc nghiệm bốn lựa chọn phù hợp với tham số đã chọn. Người dùng lần lượt trả lời từng câu, chọn đáp án và nhận phản hồi đúng/sai ngay lập tức. Kết quả của mỗi lần làm quiz được hệ thống ghi nhận vào lịch sử học tập để phục vụ cho chức năng phân tích điểm yếu.



Hình 2.6. Biểu đồ Use case luyện tập

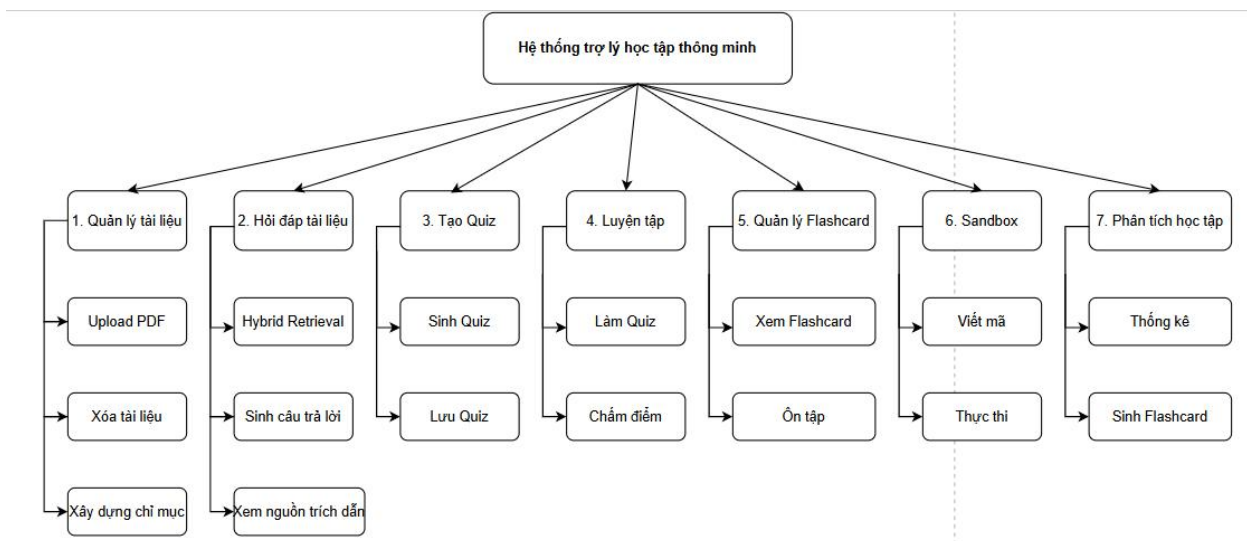
Người dùng truy cập module Luyện tập và chọn chủ đề, độ khó cùng loại bài tập mong muốn. Hệ thống sinh đề bài lập trình có cấu trúc đầy đủ gồm mô tả bài toán, yêu cầu cụ thể, định dạng đầu vào/đầu ra và ràng buộc về độ phức tạp thời gian. Người dùng đọc đề, viết bài giải vào vùng soạn thảo và nộp bài. Hệ thống tiếp nhận bài nộp và phản hồi kết quả đánh giá, giúp người dùng kiểm chứng tư duy giải thuật của mình trên từng dạng bài cụ thể.



Hình 2.7. Sơ đồ Use Case quản lý Flashcard

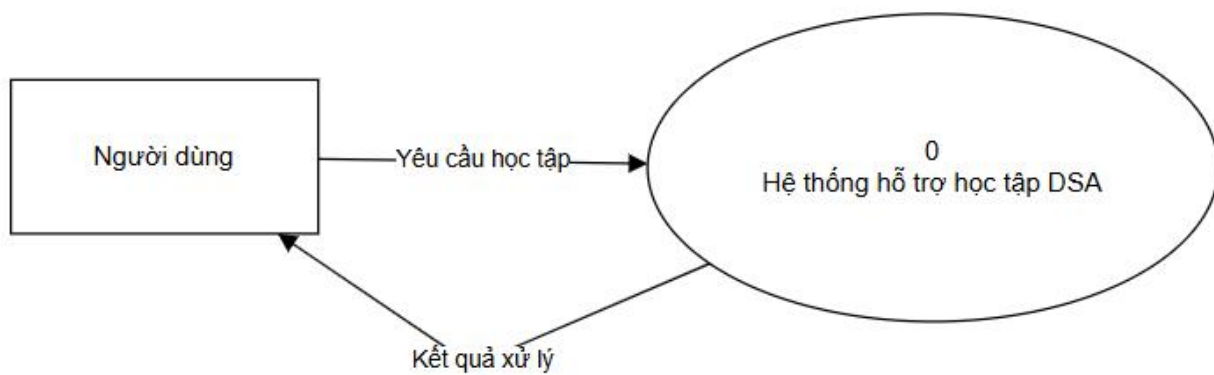
Người dùng truy cập Flashcard, chọn chủ đề muốn ôn tập và chỉ định số lượng thẻ cần tạo, sau đó nhấn nút tạo flashcard. Hệ thống tự động sinh bộ thẻ ghi nhớ với câu hỏi ở mặt trước và đáp án ở mặt sau. Người dùng lần lượt xem từng thẻ, lật thẻ để kiểm tra đáp án và điều hướng qua lại bằng các nút trước sau. Khi hoàn thành, người dùng có thể xuất toàn bộ bộ thẻ ra file định dạng Anki để tiếp tục ôn tập theo phương pháp lặp lại ngắt quãng trên phần mềm Anki.

2.4. Biểu đồ BFD

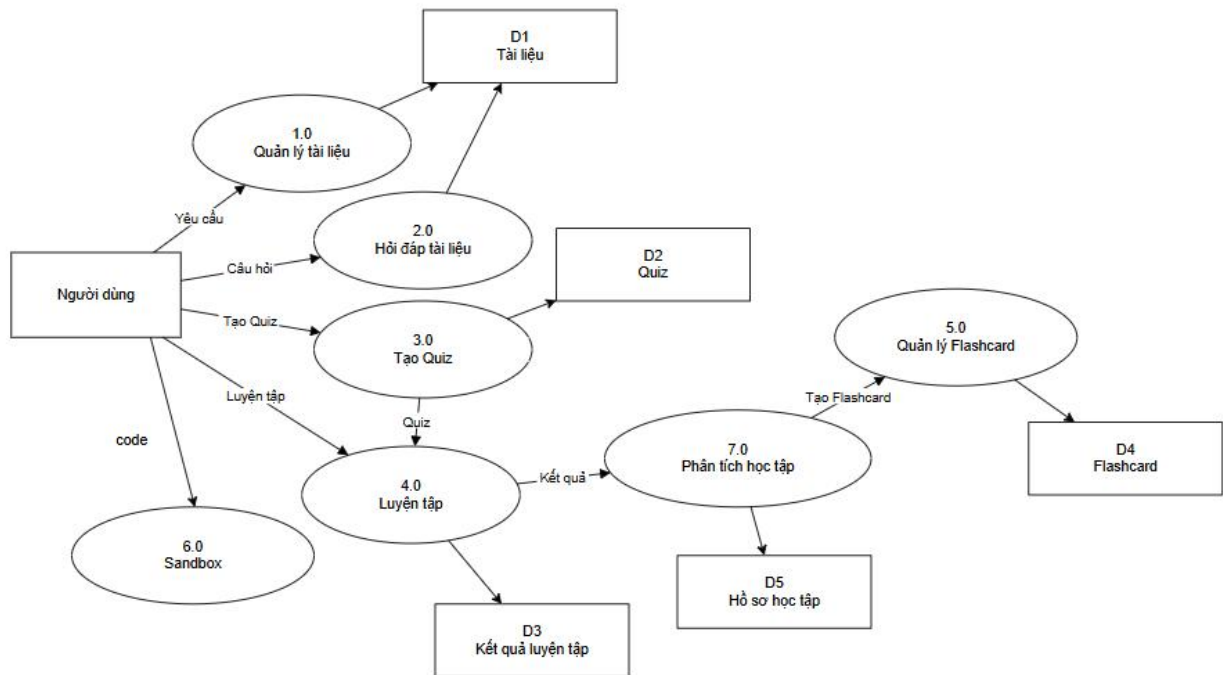


Hình 2.8. Biểu đồ BFD

2.5. Biểu đồ DFD

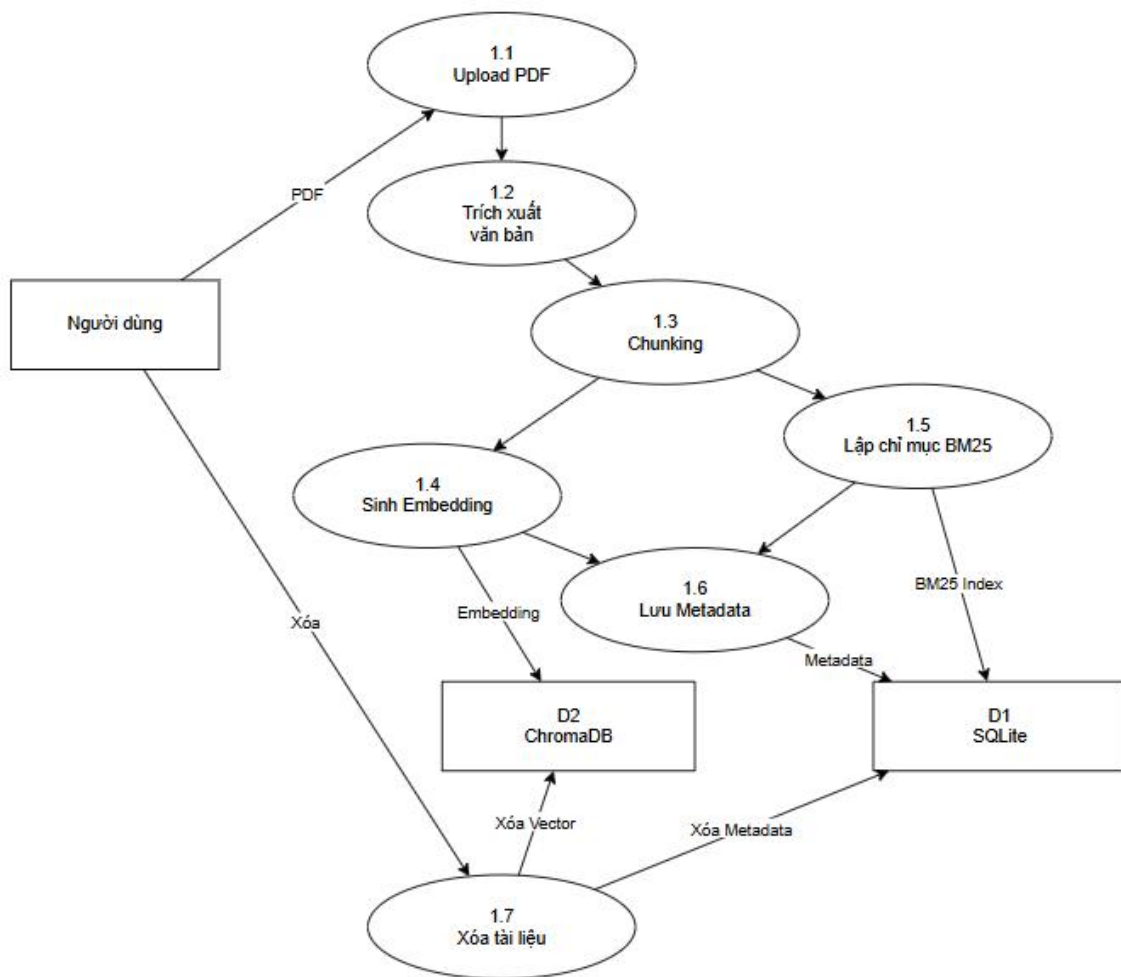


Hình 2.9. Biểu đồ DFD mức ngữ cảnh

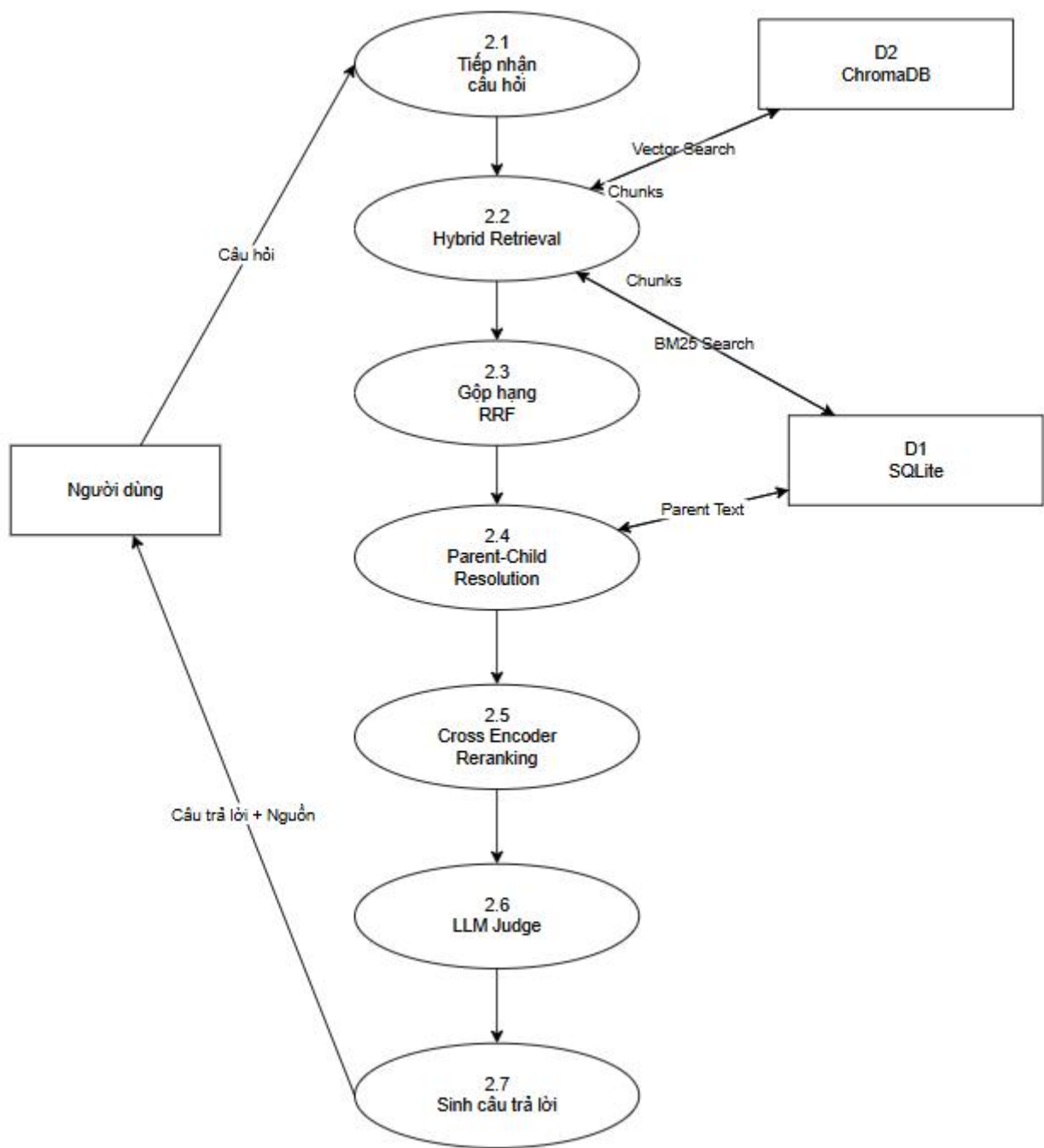


Hình 2.10. Biểu đồ DFD mức đỉnh

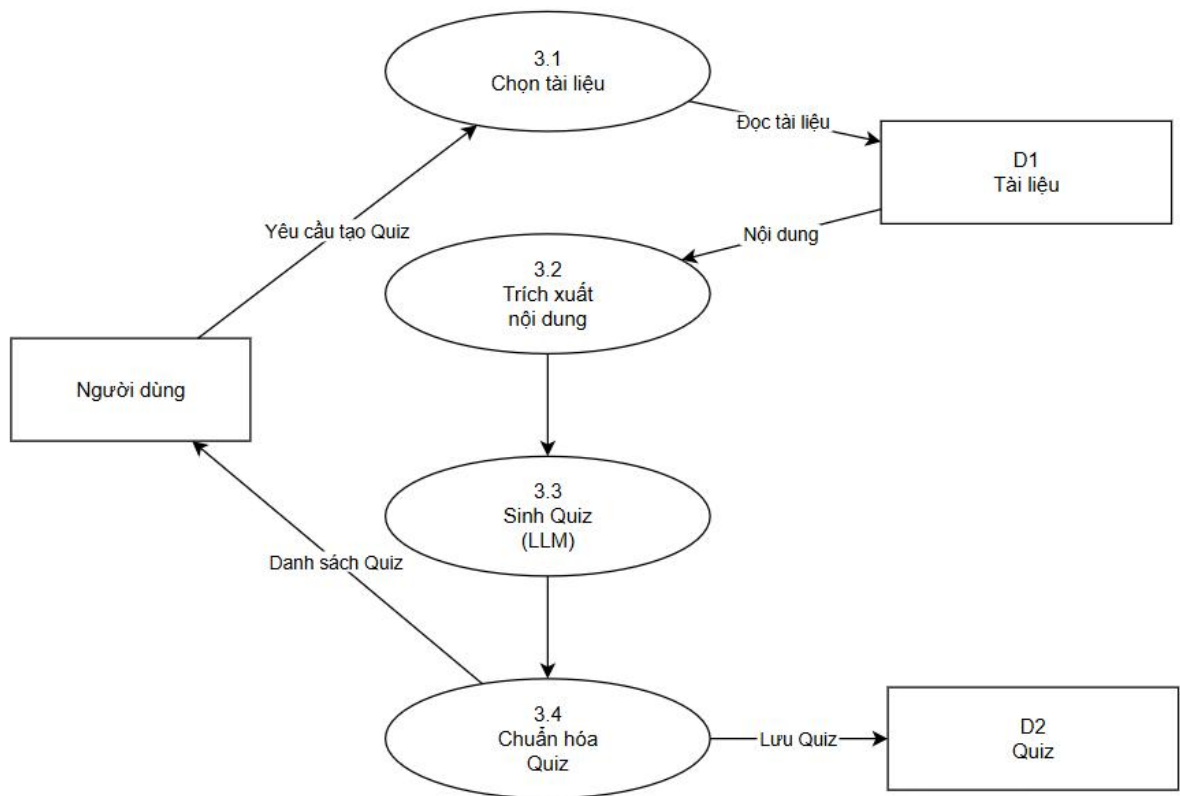
Hình 2.10 thể hiện sơ đồ luồng dữ liệu mức đỉnh của hệ thống, làm rõ mối quan hệ giữa các tiến trình xử lý và kho dữ liệu. Người dùng tương tác với hệ thống thông qua 6 tiến trình chính. Tiến trình 1.0 Quản lý tài liệu nhận yêu cầu từ người dùng và đọc/ghi dữ liệu vào kho D1 Tài liệu, đồng thời cung cấp ngữ cảnh cho tiến trình 2.0 Hỏi đáp tài liệu. Tiến trình 2.0 nhận câu hỏi từ người dùng, truy xuất tài liệu liên quan và trả về câu trả lời có trích dẫn. Tiến trình 3.0 Tạo Quiz nhận yêu cầu tạo câu hỏi từ người dùng, sinh bộ trắc nghiệm và lưu vào kho D2 Quiz, đồng thời chuyển kết quả sang tiến trình 4.0 Luyện tập. Tiến trình 4.0 Luyện tập nhận code và bài làm từ người dùng, lưu kết quả vào kho D3 Kết quả luyện tập và chuyển dữ liệu sang tiến trình 7.0 Phân tích học tập. Tiến trình 7.0 Phân tích học tập tổng hợp kết quả luyện tập, lưu hồ sơ vào kho D5 Hồ sơ học tập và tạo flashcard gửi sang tiến trình 5.0 Quản lý Flashcard để lưu vào kho D4 Flashcard. Tiến trình 6.0 Sandbox nhận code trực tiếp từ người dùng, thực thi trong môi trường cách ly và trả về kết quả mà không lưu vào kho dữ liệu nào.



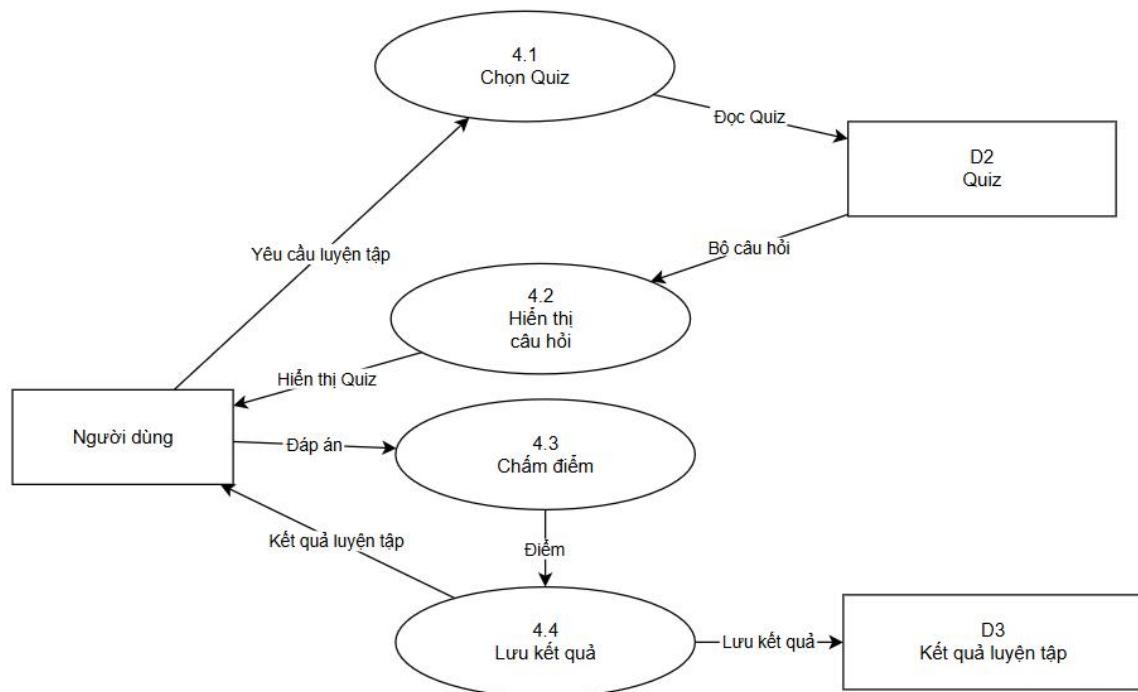
Hình 2.11. Biểu đồ DFD mức dưới đỉnh quản lý tài liệu



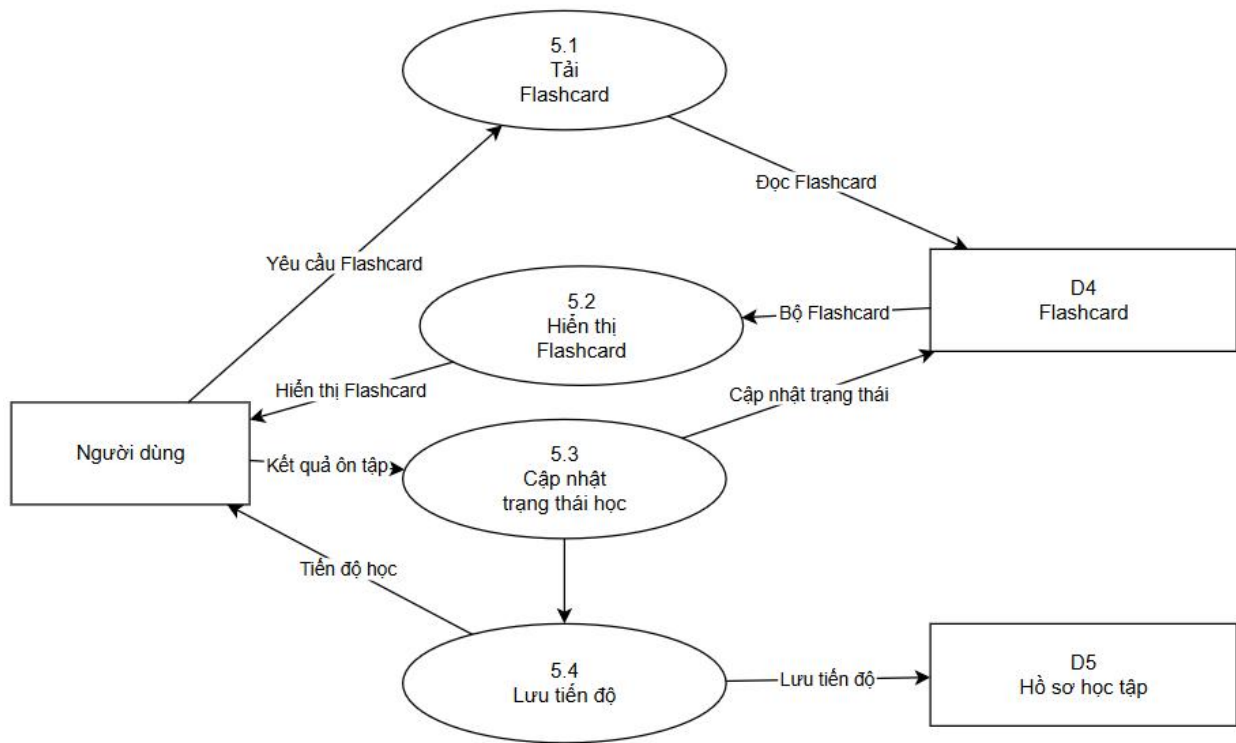
Hình 2.12. Biểu đồ DFD mức dưới đỉnh hỏi đáp tài liệu



Hình 2.13. Biểu đồ DFD mức dưới đỉnh tạo quiz



Hình 2.14. Biểu đồ DFD mức dưới đỉnh luyện tập

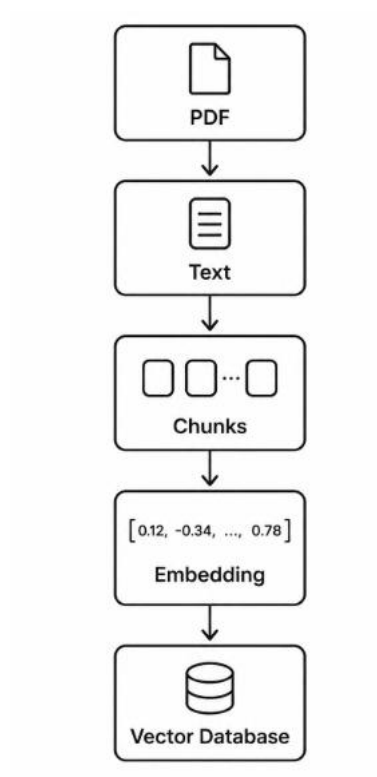


Hình 2.15. Biểu đồ DFD mức dưới đỉnh quản lý flashcard

2.6. Thiết kế và thực hiện các chiến lược Chunking

Một trong những thành phần quan trọng nhất quyết định hiệu quả của hệ thống RAG là quá trình Chunking. Đây là bước chuyển đổi tài liệu gốc có kích thước lớn thành nhiều phân đoạn nhỏ hơn trước khi tạo embedding và xây dựng chỉ mục tìm kiếm. Nếu lựa chọn kích thước hoặc phương pháp phân mảnh không phù hợp, hệ thống sẽ gặp hai vấn đề phổ biến là thông tin quan trọng bị chia cắt làm giảm khả năng truy xuất và mỗi chunk chứa quá nhiều nội dung khiến embedding bị pha loãng ngữ nghĩa.

Do đó, việc nghiên cứu và đánh giá các chiến lược chunking là nội dung trung tâm của đề tài này. Hệ thống không chỉ hiện thực nhiều phương pháp phân mảnh khác nhau mà còn xây dựng một kiến trúc thống nhất giúp dễ dàng bổ sung các thuật toán mới và tiến hành benchmark một cách khách quan.



Hình 2.16. Quá trình Chunking